Docker Explained — A Beginner's Guide

> **Module 2 | CSEC 3** This lesson introduces Docker from scratch. By the end, you should be able to install Docker, understand its core concepts, and confidently use the Docker CLI to build, run, and manage containers.

Table of Contents

1. What is Docker?
2. Why Use Docker?
3. Key Concepts
4. Installing Docker
5. Verifying Your Installation
6. Docker Images
7. Docker Containers
8. Writing a Dockerfile
9. Docker Volumes (Persistent Data)
10. Docker Networks
11. Cleaning Up
12. Cheat Sheet — Quick Reference
13. Summary

---

1. What is Docker?

Docker is an open-source platform that lets you package an application together with everything it needs to run — code, runtime, libraries, and system tools — into a single, lightweight unit called a **container**.

Think of it like a shipping container for software: no matter where you move it, the contents inside stay the same and work the same way.

Docker vs. Virtual Machines

Feature	Docker Container	Virtual Machine
Startup time	Seconds	Minutes
Size	Megabytes	Gigabytes
OS	Shares host OS kernel	Runs its own full OS
Performance	Near-native	Overhead from hypervisor
Isolation	Process-level	Full hardware-level
Portability	Very high	Moderate

> **Key Takeaway:** Containers are much lighter and faster than VMs because they share the host operating system's kernel instead of running a separate OS.

---

2. Why Use Docker?

- **Consistency** — "It works on my machine" is no longer a problem. Docker ensures the same environment everywhere.
- **Isolation** — Each container runs independently, so applications don't conflict with each other.
- **Portability** — Build once, run anywhere (laptop, server, cloud).
- **Speed** — Containers start in seconds, not minutes.
- **Scalability** — Easily spin up multiple instances of an application.
- **Reproducibility** — Dockerfiles document exactly how an environment is built, making it easy to recreate.

---

3. Key Concepts

Before diving into commands, let's understand the building blocks:

🖼️ Image

A **Docker image** is a read-only template (like a blueprint) that contains everything needed to run an application — the code, runtime, libraries, and configuration. Images are built from a **Dockerfile**.

📦 Container

A **container** is a running instance of an image. You can create multiple containers from the same image. Containers are isolated from each other and from the host system.

📄 Dockerfile

A **Dockerfile** is a text file with step-by-step instructions for building a Docker image. It defines the base OS, application code, dependencies, and startup command.

🗃️ Docker Hub

Docker Hub is a public registry (like an app store for Docker images) where you can find and share images. Official images for popular software (e.g., `nginx`, `python`, `mysql`) are available here.

💾 Volume

A **volume** is a mechanism to persist data outside the container's filesystem. Without volumes, data inside a container is lost when the container is removed.

🌐 Network

Docker **networks** allow containers to communicate with each other and with the outside world.

```
┌──────────────────────────────────────────────────────────┐
│                      Docker Engine                       │
│                                                          │
│    ┌──────────────┐    ┌──────────────┐    ┌──────────────┐      │
│    │ Container │    │ Container │    │ Container │      │
│    │  (nginx)  │    │  (python) │    │  (mysql)  │      │
│    └──────────────┘    └──────────────┘    └──────────────┘      │
│          │                  │                  │            │
│          └──────────────────┼──────────────────┘            │
│                             │                               │
│                      Docker Network                         │
└─────────────────────────────┼──────────────────────────────┘
                              │
                       Host Machine
```

---

4. Installing Docker

Docker can be installed on **Windows**, **macOS**, and **Linux**. The recommended way for Windows and macOS is **Docker Desktop**, which bundles the Docker Engine, CLI, and Docker Compose into a single installer.

4.1 Installing on Windows

System Requirements

- Windows 10 (64-bit) Build 19041 or later, or Windows 11
- 4 GB RAM minimum (8 GB recommended)
- BIOS/UEFI hardware virtualization enabled
- WSL 2 (Windows Subsystem for Linux 2) enabled

Steps

1. **Enable WSL 2** — Open PowerShell **as Administrator** and run:

   ```
   wsl --install
   ```

   Restart your computer if prompted.

2. **Download Docker Desktop** — Go to https://www.docker.com/products/docker-desktop and download the Windows installer.

3. **Run the Installer** — Double-click `Docker Desktop Installer.exe`. During installation, make sure the option **"Use WSL 2 instead of Hyper-V"** is checked.

4. **Restart your computer** if prompted.
5. **Launch Docker Desktop** — Open Docker Desktop from the Start Menu. It may take a minute to start the Docker Engine.
6. **Accept the license agreement** on first launch.

4.2 Installing on macOS

System Requirements

- macOS Mojave (10.14) or later
- Apple Silicon (M1/M2/M3/M4) or Intel processor

Steps

1. **Download Docker Desktop** — Go to https://www.docker.com/products/docker-desktop and download the Mac installer.
    - Choose **Apple Silicon** if you have an M-series chip, or **Intel** for older Macs.
2. **Install** — Open the `.dmg` file and drag the Docker icon into your **Applications** folder.
3. **Launch Docker Desktop** — Open **Docker.app** from your Applications folder.
4. **Grant permissions** — Allow Docker to install its helper tools when prompted.
5. A 🐳 whale icon will appear in your menu bar when Docker is running.

4.3 Installing on Linux (Ubuntu/Debian)

On Linux, you install the **Docker Engine** directly (no Desktop GUI needed).

Step 1: Remove old versions (if any)

```
sudo apt-get remove docker docker-engine docker.io containerd runc
```

Step 2: Update packages and install prerequisites

```
sudo apt-get update
sudo apt-get install ca-certificates curl gnupg
```

Step 3: Add Docker's official GPG key

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

Step 4: Add the Docker repository

```
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Step 5: Install Docker Engine

```
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
```

Step 6: (Optional) Run Docker without `sudo`

```
sudo usermod -aG docker $USER
```

> **Note:** Log out and log back in for this to take effect.

---

5. Verifying Your Installation

After installation, open your terminal (Command Prompt, PowerShell, or Terminal) and run:

Check the Docker version

```
docker --version
```

Expected output:

```
Docker version 27.x.x, build xxxxxxx
```

Run the hello-world test container

```
docker run hello-world
```

Expected output:

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
...
```

> **What just happened?**
>
> 1. Docker looked for the `hello-world` image locally.
> 2. It didn't find it, so it **pulled** the image from Docker Hub.
> 3. It created a **container** from that image.
> 4. The container ran, printed the message, and then stopped.

View system-wide information

```
docker info
```

This shows details about your Docker installation: number of containers, images, storage driver, OS, and more.

---

6. Docker Images

Images are the foundation of containers. Let's learn how to manage them.

6.1 Pulling an Image

Download an image from Docker Hub:

```
docker pull <image_name>:<tag>
```

Example — Pull the latest Ubuntu image:

```
docker pull ubuntu:latest
```

Example — Pull a specific Nginx version:

```
docker pull nginx:1.25
```

> **Explanation:** The `<tag>` specifies a version. If you omit the tag, Docker defaults to `latest`.

6.2 Listing Local Images

See all images stored on your machine:

```
docker images
```

Sample output:

```
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
ubuntu        latest    3db8720ecbf5   2 weeks ago    77.9MB
nginx         1.25      a8758716bb6a   3 weeks ago    187MB
hello-world   latest    d2c94e258dcb   10 months ago  13.3kB
```

6.3 Removing an Image

Delete an image you no longer need:

```
docker rmi <image_name>:<tag>
```

Example:

```
docker rmi hello-world:latest
```

> **Note:** You can't remove an image if a container (even a stopped one) is using it. Remove the container first.

6.4 Searching for Images

Search Docker Hub from the CLI:

```
docker search <keyword>
```

Example:

```
docker search python
```

---

7. Docker Containers

Containers are where the action happens. Here are the essential commands.

7.1 Running a Container

The `docker run` command creates and starts a container from an image.

```
docker run [OPTIONS] <image_name> [COMMAND]
```

Example 1: Run Ubuntu interactively

```
docker run -it ubuntu bash
```

Flag	Meaning
`-i`	Keep STDIN open (interactive)
`-t`	Allocate a pseudo-TTY (terminal)

This drops you into an Ubuntu shell inside the container. Type `exit` to leave.

Example 2: Run Nginx in the background

```
docker run -d --name my-nginx -p 8080:80 nginx
```

Flag	Meaning
`-d`	Detached mode (run in background)
`--name my-nginx`	Give the container a friendly name
`-p 8080:80`	Map port 8080 on the host to port 80 in the container

Now open `http://localhost:8080` in your browser — you'll see the Nginx welcome page! 🎉

Example 3: Run with automatic cleanup

```
docker run --rm ubuntu echo "Hello from Docker!"
```

Flag	Meaning
`--rm`	Automatically remove the container when it exits

7.2 Listing Containers

Show running containers:

```
docker ps
```

Show all containers (including stopped):

```
docker ps -a
```

Sample output:

```
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS
PORTS                  NAMES
a1b2c3d4e5f6   nginx     "/docker-entrypoint.…"   5 minutes ago    Up 5 minutes
0.0.0.0:8080->80/tcp   my-nginx
f6e5d4c3b2a1   ubuntu    "bash"                   10 minutes ago   Exited (0)
loving_darwin
```

7.3 Stopping a Container

```
docker stop <container_name_or_id>
```

Example:

```
docker stop my-nginx
```

7.4 Starting a Stopped Container

```
docker start <container_name_or_id>
```

Example:

```
docker start my-nginx
```

7.5 Restarting a Container

```
docker restart <container_name_or_id>
```

7.6 Removing a Container

Remove a **stopped** container:

```
docker rm <container_name_or_id>
```

Force remove a **running** container:

```
docker rm -f <container_name_or_id>
```

Example:

```
docker rm my-nginx
```

7.7 Executing Commands Inside a Running Container

Use `docker exec` to run a command inside a container that is already running:

```
docker exec -it <container_name_or_id> <command>
```

Example — Open a bash shell inside the running Nginx container:

```
docker exec -it my-nginx bash
```

Once inside, you can run commands like `ls`, `cat`, etc. Type `exit` to leave.

7.8 Viewing Container Logs

See what a container has been printing to the console:

```
docker logs <container_name_or_id>
```

Follow logs in real-time (like `tail -f`):

```
docker logs -f <container_name_or_id>
```

7.9 Inspecting a Container

Get detailed information about a container in JSON format:

```
docker inspect <container_name_or_id>
```

---

8. Writing a Dockerfile

A **Dockerfile** is a recipe for building your own custom image. Each instruction creates a **layer** in the image.

8.1 Common Dockerfile Instructions

Instruction	Purpose	Example
`FROM`	Set the base image	`FROM python:3.12-slim`
`WORKDIR`	Set the working directory inside the container	`WORKDIR /app`
`COPY`	Copy files from host to container	`COPY . /app`
`RUN`	Execute a command during image build	`RUN pip install flask`
`EXPOSE`	Document which port the app uses	`EXPOSE 5000`
`CMD`	Default command when container starts	`CMD ["python", "app.py"]`
`ENV`	Set environment variables	`ENV APP_ENV=production`

8.2 Example: Simple Python Web App

Let's create a tiny Flask web application and containerize it.

Step 1: Create the project files

`app.py`

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def hello():
    return "<h1>Hello from Docker!</h1><p>This app is running inside a container.
</p>"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

`requirements.txt`

```
flask
```

Step 2: Create the Dockerfile

`Dockerfile` (no file extension)

```
# 1. Start from a lightweight Python base image
FROM python:3.12-slim

# 2. Set the working directory inside the container
WORKDIR /app

# 3. Copy the requirements file first (for better caching)
COPY requirements.txt .

# 4. Install Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# 5. Copy the rest of the application code
COPY . .

# 6. Document the port the application uses
EXPOSE 5000

# 7. Define the default command to run the application
CMD ["python", "app.py"]
```

> **Why copy `requirements.txt` first?** Docker caches each layer. If your code changes but `requirements.txt` stays the same, Docker won't reinstall dependencies — making rebuilds much faster.

Step 3: Build the image

```
docker build -t my-flask-app .
```

Part	Meaning
`docker build`	Build an image from a Dockerfile
`-t my-flask-app`	Tag (name) the image as `my-flask-app`
`.`	Use the current directory as the build context

Step 4: Run the container

```
docker run -d --name flask-container -p 5000:5000 my-flask-app
```

Now visit `http://localhost:5000` in your browser — you should see **"Hello from Docker!"** 🎉

Step 5: Stop and clean up

```
docker stop flask-container
docker rm flask-container
```

8.3 Example: Simple Nginx Static Website

You can also use Docker to serve a static HTML website with Nginx.

Step 1: Create the project files

`index.html`

```
<!DOCTYPE html>
<html>
<head>
    <title>My Docker Website</title>
</head>
<body>
    <h1>Welcome to My Dockerized Website!</h1>
    <p>This page is being served by Nginx inside a Docker container.</p>
```

```
</body>
</html>
```

Step 2: Create the Dockerfile

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

Step 3: Build and run

```
docker build -t my-website .
docker run -d --name website -p 8080:80 my-website
```

Visit `http://localhost:8080` to see your website! 🌐

---

9. Docker Volumes (Persistent Data)

By default, data inside a container is **temporary** — it disappears when the container is removed. **Volumes** solve this problem by storing data outside the container.

9.1 Why Volumes?

Without Volumes	With Volumes
Data is lost when container is removed	Data persists across container restarts and removals
Data can't be shared between containers	Volumes can be shared between multiple containers

9.2 Creating and Using Volumes

Create a named volume:

```
docker volume create my-data
```

Run a container with a volume attached:

```
docker run -d --name my-db \
  -v my-data:/var/lib/mysql \
```

```
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
```

Part	Meaning
`-v my-data:/var/lib/mysql`	Mount the volume `my-data` to `/var/lib/mysql` inside the container
`-e MYSQL_ROOT_PASSWORD=secret`	Set an environment variable

> **Explanation:** Now, even if you remove and recreate the MySQL container, your database data is safe inside the `my-data` volume.

9.3 Bind Mounts (Host Directory Mapping)

Instead of a named volume, you can map a directory on your host machine directly:

```
docker run -d --name my-app \
  -v /path/on/host:/app/data \
  my-image
```

Example — Live-edit a website:

```
docker run -d --name dev-site \
  -v $(pwd)/html:/usr/share/nginx/html \
  -p 8080:80 \
  nginx
```

Now any changes you make to the `html/` folder on your host are immediately reflected in the container!

9.4 Volume Management Commands

```
# List all volumes
docker volume ls

# Inspect a volume (see where it's stored, etc.)
docker volume inspect my-data

# Remove a volume
docker volume rm my-data

# Remove all unused volumes
docker volume prune
```

---

10. Docker Networks

Docker networks let containers talk to each other. Docker creates three default networks automatically:

Network	Driver	Description
`bridge`	bridge	Default for standalone containers
`host`	host	Shares the host's network (no isolation)
`none`	null	No networking at all

10.1 Creating a Custom Network

```
docker network create my-network
```

10.2 Connecting Containers to a Network

Option A: At run time

```
docker run -d --name app --network my-network my-app-image
docker run -d --name db --network my-network mysql:8.0
```

Now the `app` container can reach the `db` container by its name (`db`) as the hostname.

Option B: Connect an existing container

```
docker network connect my-network existing-container
```

10.3 Example: App + Database Communication

```
# Create a network
docker network create app-net

# Start a MySQL database
docker run -d --name my-db \
  --network app-net \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=myapp \
  mysql:8.0

# Start an application that connects to the database
docker run -d --name my-app \
```

```
  --network app-net \
  -p 3000:3000 \
  -e DB_HOST=my-db \
  -e DB_PORT=3306 \
  my-app-image
```

> **Explanation:** Both containers are on the `app-net` network. The app can connect to MySQL using the hostname `my-db` (the container name) — no IP addresses needed!

10.4 Network Management Commands

```
# List all networks
docker network ls

# Inspect a network
docker network inspect my-network

# Disconnect a container from a network
docker network disconnect my-network my-container

# Remove a network
docker network rm my-network

# Remove all unused networks
docker network prune
```

---

11. Cleaning Up

Over time, Docker can accumulate unused images, containers, volumes, and networks. Here's how to clean up:

Remove stopped containers

```
docker container prune
```

Remove unused images

```
docker image prune
```

Remove **all** images not used by any container:

```
docker image prune -a
```

Remove unused volumes

```
docker volume prune
```

Remove unused networks

```
docker network prune
```

🧹 Nuclear option — Remove everything unused

```
docker system prune -a --volumes
```

> ⚠️ **Warning:** This removes ALL stopped containers, ALL unused networks, ALL unused images, and ALL unused volumes. Use with caution!

---

12. Cheat Sheet — Quick Reference

Images

Command	Description
`docker pull <image>:<tag>`	Download an image
`docker images`	List local images
`docker rmi <image>`	Remove an image
`docker build -t <name> .`	Build image from Dockerfile
`docker search <keyword>`	Search Docker Hub

Containers

Command	Description
`docker run -d --name <n> -p <h>:<c> <image>`	Create & start a container
`docker run -it <image> bash`	Run interactively

Command	Description
`docker ps`	List running containers
`docker ps -a`	List all containers
`docker stop <container>`	Stop a container
`docker start <container>`	Start a stopped container
`docker restart <container>`	Restart a container
`docker rm <container>`	Remove a stopped container
`docker exec -it <container> bash`	Shell into container
`docker logs <container>`	View container logs
`docker inspect <container>`	Detailed container info

Volumes

Command	Description
`docker volume create <name>`	Create a volume
`docker volume ls`	List volumes
`docker volume rm <name>`	Remove a volume
`docker volume inspect <name>`	Inspect a volume
`docker volume prune`	Remove unused volumes

Networks

Command	Description
`docker network create <name>`	Create a network
`docker network ls`	List networks
`docker network connect <net> <container>`	Connect container to network
`docker network disconnect <net> <container>`	Disconnect container
`docker network rm <name>`	Remove a network
`docker network prune`	Remove unused networks

System

Command	Description
`docker --version`	Show Docker version
`docker info`	System-wide information
`docker system prune`	Clean up unused resources
`docker system prune -a --volumes`	Deep clean everything

---

13. Summary

In this lesson, you learned:

Topic	What You Learned
What is Docker	A platform for packaging apps into portable containers
Installation	How to install Docker on Windows, macOS, and Linux
Images	How to pull, list, build, and remove images
Containers	How to run, stop, start, remove, and interact with containers
Dockerfiles	How to write a Dockerfile to create custom images
Volumes	How to persist data using volumes and bind mounts
Networks	How to enable container-to-container communication
Cleanup	How to remove unused Docker resources

What's Next?

- **Docker Compose** — Define and run multi-container applications with a single YAML file.
- **Docker Swarm / Kubernetes** — Orchestrate containers at scale.
- **Docker Security** — Best practices for securing your containers.
- **CI/CD with Docker** — Automate building and deploying containers.

> 📝 **Practice Exercise:** Try to containerize a simple project of your own! Create a `Dockerfile`, build the image, and run it. Experiment with volumes and networks.